Práctica final

Pascual Rincón Cardona

Simón Foguel Giraldo

Juan Pablo Vega Ruiz

Escuela de Ciencias Aplicadas e Ingeniería, Universidad EAFIT

Estructuras de Datos y Algoritmos

Docente: Carlos Alberto Álvarez

30 de octubre de 2026

Introducción

En el campo de la computación existe una gran cantidad de algoritmos para organizar conjuntos de datos con base en un criterio de comparación. Cada uno tiene ventajas y desventajas que dependen de la naturaleza de los datos, el hardware, el consumo de memoria y, sobre todo, la complejidad algorítmica. En el presente trabajo se analizan tres métodos de ordenamiento implementados en C++: el ordenamiento mediante un árbol AVL, Heapsort y Quicksort (Cormen et al., 2022). Se comparan los tiempos de ejecución al ordenar un mismo conjunto de 100.000 palabras dispuestas de forma aleatoria. Posteriormente, se analizan los resultados con base en la complejidad temporal, el manejo de memoria, el uso eficiente de la caché y el hardware, y se determina cuál enfoque resulta más apropiado para este problema.

Conjunto de datos

El archivo con las palabras fue suministrado por el docente y contiene 100.000 palabras ya dispuestas en orden aleatorio. Se verificó que no existen palabras repetidas, condición en la que se apoyan varias decisiones de implementación. La palabra más larga tiene 29 caracteres, dato relevante porque permite acotar el costo de comparar dos cadenas por una constante, como se explica en el Análisis.

Implementación

Árbol AVL

La implementación del árbol AVL sigue la formulación tradicional donde cada nodo almacena la palabra, las direcciones de sus dos hijos y un campo de altura que permite calcular el factor de balance. El subárbol izquierdo contiene las palabras menores que la del nodo y el derecho, las mayores. Tras cada inserción, el árbol retrocede de forma recursiva hacia la raíz, actualiza las alturas y aplica rotaciones simples o dobles cuando el factor de balance, definido

como la altura del subárbol derecho menos la del izquierdo, se sale del intervalo [−1, 1]. Dado que no hay palabras repetidas en la base de datos, no se añadió un conteo de apariciones: si se intenta insertar un elemento ya presente, la inserción lo ignora y retorna falso. La estructura implementa los constructores y operadores de asignación por copia y por movimiento, además de la inserción de elementos por copia y por movimiento. Para obtener las palabras ordenadas se realiza un recorrido *inorder*, apoyado en una pila implementada sobre un vector de punteros a nodos, que va insertando las palabras en un vector de salida.

Heapsort

Se usó un vector de la biblioteca estándar para implementar el *binary heap*, dado que permite un acceso inmediato a los elementos mediante aritmética de punteros (Cormen et al., 2022); una estructura de nodos enlazados, debido al manejo de punteros, supondría una pérdida considerable de eficiencia. Se crearon varios constructores, entre ellos uno que recibe un vector con elementos y, mediante la función *heapify*, basada en el algoritmo de Floyd, lo transforma en un *binary heap* en O(n). Se decidió implementarlo en base 0, lo cual implica cambios en el cálculo de los índices de hijos y padres, que para un nodo en la posición i son los de la Ecuación 1.

$$h_{izq}(i) = 2i + 1, \quad h_{der}(i) = 2i + 2, \quad p(i) = \left\lfloor \frac{i-1}{2} \right\rfloor \tag{1}$$

Dado que se pretende organizar de forma ascendente el conjunto de datos, se construye un *max heap*: para ordenar el vector subyacente se mueve la raíz, es decir, el primer elemento, a la última posición del arreglo y se restablece el heap en el vector sin contar el último elemento, procediendo así hasta haber ordenado todo el arreglo (Williams, 1964). Posteriormente, el vector se retorna mediante semántica de movimiento; esto implica que el *binary heap* queda inutilizable, pero se hace así para maximizar la eficiencia.

Quicksort

Quicksort se implementó con base en el esquema de partición de Lomuto (Cormen et al., 2022), en el que el pivote se toma como el último elemento de cada partición. En este caso, dicho esquema no resulta problemático debido a la naturaleza de los datos (sin elementos repetidos y en orden aleatorio), como se explica en el análisis de complejidad. El algoritmo recibe el vector por referencia y realiza el ordenamiento sobre él, evitando copias innecesarias.

Memoria

Árbol AVL

Es el algoritmo que más memoria requiere, dado que se basa en una estructura que debe llevar registro de múltiples punteros. Para 100.000 elementos se crean 100.000 nodos, cada uno con la palabra, dos punteros a sus hijos y la altura. Mediante el operador `sizeof` se obtuvo que la gestión de los nodos requiere un total de 5.600.016 bytes, es decir, 56 bytes por nodo más 16 bytes adicionales correspondientes al objeto del árbol (puntero a la raíz, contador de elementos y relleno) (Ecuaciones 2 y 3).

$$M_{nodo} = 32 + 8 + 8 + 4 + 4 = 56\,B \tag{2}$$

$$M_{AVL} = 100.000 \times 56 + 16 = 5.600.016\,B \tag{3}$$

En la Ecuación 2, los 32 bytes corresponden a la cadena `std::string`, 8 + 8 a los punteros a los hijos, y 4 + 4 a la altura y al relleno (*padding*). El tamaño de 32 bytes obtenido para la cadena corresponde a cadenas de menos de 15 caracteres, que son almacenados directamente dentro de ella, gracias a la optimización de cadenas pequeñas (SSO). Como la palabra más larga del conjunto tiene 29 caracteres, algunas palabras superan ese límite y requieren una asignación dinámica adicional, por lo que el uso real de memoria puede ser mayor; aun así, el consumo del árbol es O(n). Para ordenar la lista se realiza un recorrido apoyado en

una pila de punteros a nodos, con uso de memoria O(log n), y los elementos se almacenan en un vector de salida, que requiere O(n). En total, el árbol necesita cerca de 5,6 MB para los nodos y 3,2 MB para el vector de salida (Tabla 1). Más que la cantidad de memoria en sí, lo que probablemente compromete el rendimiento es la forma en que se utiliza: 100.000 asignaciones dinámicas independientes y un recorrido que debe seguir punteros entre nodos dispersos.

Heapsort

Heapsort hace uso de un *binary heap* implementado sobre un vector, lo que reduce el consumo de memoria frente al árbol, ya que solo se almacena el elemento y no punteros adicionales. Requiere cerca de 3.200.000 bytes (100.000 × 32 bytes), dado que la mayoría de las palabras tiene menos de 15 caracteres. El consumo de memoria es linealmente proporcional al número de elementos y no requiere memoria adicional para el proceso de ordenamiento, es decir, O(1) de espacio auxiliar.

Quicksort

Quicksort se basa en un vector sobre el cual se ordenan los elementos. Su uso de memoria es el mismo que el de Heapsort: un vector de 100.000 cadenas. Por ser un algoritmo recursivo, realiza múltiples llamadas recursivas con una profundidad aproximada de log n, por lo cual el proceso de ordenación requiere O(log n) de espacio adicional en la pila de llamadas. Este valor es válido solo en promedio, dado que con el pivote en el último elemento, la profundidad de la recursión puede llegar a O(n) en el peor caso. Esto se corrige recursando primero sobre la partición más pequeña y eliminando la recursión sobre la más grande, lo que acota la profundidad a O(log n) en todos los casos (Cormen et al., 2022). La implementación presentada no aplica esta optimización, por lo que queda expuesta a ese peor caso, aunque con datos aleatorios y sin repetidos es muy improbable que ocurra.

Tabla 1

Memoria aproximada requerida por cada algoritmo para 100.000 palabras

Algoritmo	Estructura principal	Memoria aproximada	Espacio auxiliar
AVL Tree	100.000 nodos de 56 B más vector de salida	≈ 5,6 MB + 3,2 MB = 8,8 MB	O(log n) (pila)
HeapSort	Vector de 100.000 cadenas	≈ 3,2 MB	O(1)
QuickSort	Vector de 100.000 cadenas	≈ 3,2 MB	O(log n)*

Nota. Valores calculados con `sizeof(std::string)` = 32 bytes; no incluyen la memoria dinámica de las palabras de más de 15 caracteres. * En el peor caso O(n), poco probable por la naturaleza de los datos.

Mediciones

Para evitar ruido en las mediciones como la lectura del archivo (entrada y salida), se cargaron todos los datos a un vector antes de empezar a medir. El tiempo cronometrado comprende únicamente la ejecución del algoritmo sobre dicho vector (en el caso del árbol, la inserción de todos los elementos mediante un ciclo y el recorrido *inorder*) y la generación del vector con los datos organizados de forma ascendente, momento en el cual se detiene el cronómetro. En cada medición, el vector se vuelve a cargar desde el archivo antes de iniciar el cronómetro, de modo que cada algoritmo parte de una copia idéntica de los datos originales. Se realizó un total de 5 mediciones por algoritmo y se calcularon el promedio y la desviación estándar muestral. La correcta ordenación de la salida de cada algoritmo se verificó mediante un programa aparte.

Tabla 2

Tiempo de ejecución de cada algoritmo para 100.000 palabras (µs)

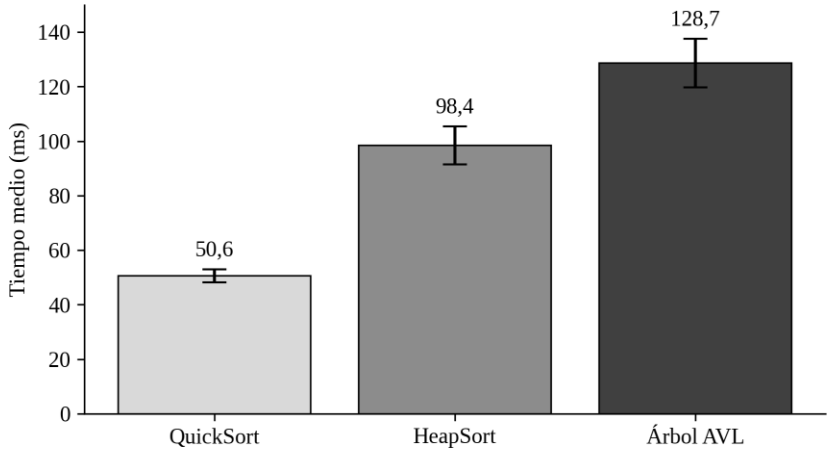
Algoritmo	Medición 1	Medición 2	Medición 3	Medición 4	Medición 5	Media	Desv. est.
QuickSort	54.578	49.291	51.078	49.310	48.755	50.602,4	2.389,28
HeapSort	91.478	108.481	92.279	98.881	101.104	98.444,6	6.973,89
AVL Tree	123.590	142.492	120.396	124.606	132.238	128.664,4	8.867,92

Nota. Tiempo medido en microsegundos (µs). La desviación estándar es muestral (divisor k − 1, con k = 5).

Quicksort fue el algoritmo más rápido, con una media de 50.602,4 µs, seguido por Heapsort (98.444,6 µs) y por el árbol AVL (128.664,4 µs). Como muestra la Ecuación 7, Quicksort resultó aproximadamente 1,9 veces más rápido que Heapsort y 2,5 veces más rápido que el árbol AVL, mientras que este último fue cerca de 1,3 veces más lento que Heapsort. En cuanto a la variabilidad, Quicksort presenta la menor desviacion, seguido por el heapsort y finalmente el árbol AVL con la mayor desviación.

Figura 1

Tiempo medio de ejecución por algoritmo



Nota. Las barras de error representan una desviación estándar muestral (k = 5).

Análisis

Para el análisis se toma la comparación entre cadenas como O(1), a pesar de que en realidad es O(L), donde L es la longitud de las cadenas. Esto se debe a que la comparación está acotada por la longitud de la palabra más larga de la lista (29 caracteres), es decir, una cantidad constante de pasos. La Tabla 3 resume la complejidad de cada algoritmo.

Tabla 3

Complejidad temporal y espacial de cada algoritmo

Algoritmo	Mejor caso	Caso promedio	Peor caso	Espacio auxiliar
QuickSort	O(n log n)	O(n log n)	$O(n^2)$	O(log n)*
HeapSort	O(n log n)	O(n log n)	O(n log n)	O(1)
AVL Tree	O(n log n)	O(n log n)	O(n log n)	O(n)

Nota. n = número de palabras. * Caso promedio.

Con base en los resultados obtenidos, se puede afirmar que el algoritmo con mejor desempeño fue Quicksort, seguido por Heapsort y, finalmente, el árbol AVL. En primer lugar, ¿por qué difieren los resultados experimentales a pesar de que los tres tienen la misma complejidad en el mejor caso y en el caso promedio? Por definición, al hacer uso de la notación asintótica se pueden despreciar factores numéricos y, al tener la suma de dos funciones, se puede considerar únicamente la mayor. Estas propiedades permiten determinar de qué manera cambia el comportamiento de un algoritmo a medida que aumenta el número de datos, pero no revelan el tiempo de ejecución real, que depende de las constantes ocultas, de la cantidad de operaciones y del acceso a memoria.

Organizar mediante un árbol implica realizar la inserción de todos los elementos para, posteriormente, efectuar un recorrido *inorder*, con una complejidad de O(n log n) para todas las inserciones y O(n) para el recorrido.

Además, la cantidad de operaciones que se realizan por debajo de esa expresión resulta considerable en comparación con los otros algoritmos. Tras cada inserción, el árbol debe retroceder de forma recursiva, actualizando las alturas de los nodos y realizando rotaciones simples o dobles, que implican reasignaciones de punteros. Además, la creación de cada nodo implica una asignación de memoria dinámica y la inicialización de sus miembros, una operación considerablemente más costosa que escribir en una posición de un vector.

El modelo de complejidad asume que el acceso a memoria cuesta O(1), lo cual es una simplificación: el tiempo de acceso depende del nivel de la jerarquía de memoria en el que se encuentre el dato (Hennessy & Patterson, 2019). Cuando se accede a una dirección, se traen a la caché bloques contiguos de memoria, por lo cual, si se accede después a un elemento cercano, es probable que este ya se encuentre en la caché. Los fallos de caché deterioran de forma considerable el rendimiento, por lo que el comportamiento de la caché influye de forma notable en el rendimiento de los algoritmos de ordenamiento. En el árbol AVL, esta localidad probablemente se pierde, ya que los nodos se reservan dinámicamente en el orden de llegada de las palabras, que es aleatorio, de modo que el recorrido *inorder*, que visita los nodos en orden alfabético, salta entre direcciones de memoria distantes y es probable que genere numerosos fallos de caché, obligando al procesador a recurrir a la memoria RAM. A esto se suma que las palabras de más de 15 caracteres requieren asignación de memoria dinámica. La asignación dinámica, además, obliga al sistema a buscar bloques de memoria libres y puede producir fragmentación.

En cuanto a Heapsort, la construcción del heap mediante el algoritmo de Floyd reduce el costo de esa fase de O(n log n) a O(n) (Cormen et al., 2022); posteriormente, se reorganiza el arreglo interno en O(n log n).

A pesar de tener la misma complejidad que la ordenación mediante el árbol AVL, Heapsort tiene un costo mucho menor por operación, dado que no tiene que realizar balanceos de ningún tipo. Además, esta estructura evita el uso de nodos y, por ende, la gestión y asignación de memoria dinámica para cada elemento: solo debe manejar un bloque de memoria contiguo que permite el acceso inmediato por aritmética de índices. El ahorro de memoria también es

considerable (Tabla 1), dado que cada nodo del árbol debía almacenar el dato, los punteros a sus hijos y la altura, mientras que ahora solo se almacena el elemento.

Aun así, este algoritmo tiene menor rendimiento que Quicksort, por dos razones principales. La primera es su patrón de acceso a memoria: al restablecer la propiedad de heap, el algoritmo salta entre posiciones del arreglo cada vez más distantes, ya que los hijos del nodo en la posición i se encuentran en las posiciones 2i + 1 y 2i + 2, lo que desaprovecha la localidad de la caché, sobre todo para grandes volúmenes de datos (LaMarca & Ladner, 1999). El cálculo aritmético de los índices en sí es muy barato, por lo que no es la causa de la diferencia. La segunda razón es que Heapsort realiza más trabajo por elemento: por cada nivel de la función que permite restablecer el heap, se efectúan alrededor de dos comparaciones entre cadenas, además de un mayor número de intercambios. En promedio, Quicksort realiza aproximadamente 1,39 n $\log_2$ n comparaciones, frente a cerca de 2 n $\log_2$ n de Heapsort (Sedgewick & Wayne, 2011), como se expresa en la Ecuación 4.

$$C_Q \approx 1{,}39\, n\, log_2\, n, \quad C_H \approx 2\, n\, log_2\, n \tag{4}$$

Para n = 100.000 (con $\log_2$ n ≈ 16,61), esto equivale a unas 2,31 × 10^6 comparaciones para Quicksort y 3,32 × 10^6 para Heapsort, es decir, cerca de un 30 % menos. Esta diferencia por sí sola no explica los la diferencia considerable de rendimiento, por lo que el resto se atribuye probablemente al patrón de acceso de memoria.

En cuanto a Quicksort, fue el más eficiente de los tres. Se implementó con base en la partición de Lomuto, donde siempre se escoge el último elemento de cada una de las particiones. Su ventaja es su fácil implementación, pero su desempeño puede deteriorarse cuando existen muchos elementos duplicados o parcialmente ordenados (Cormen et al., 2022). En este caso, debido a la ausencia de elementos repetidos y a su distribución aleatoria, es muy improbable que

se presenten particiones degeneradas, y en la mayoría de los casos Quicksort divide los subproblemas en particiones razonablemente balanceadas, con un tiempo esperado de O(n log n) (Ecuación 5). En el peor caso, en cambio, una de las particiones queda vacía en cada paso y el tiempo es $O(n^2)$ (Ecuación 6).

$$T_{prom}(n) = 2T\left(\frac{n}{2}\right) + \Theta(n) = \Theta(n \log n) \quad (5)$$

$$T_{peor}(n) = T(n-1) + \Theta(n) = \Theta(n^2) \quad (6)$$

Dependiendo de la naturaleza de los datos, Heapsort podría ser una mejor opción, o un esquema diferente de Quicksort, como el de Hoare o la mediana de tres, que reducen la probabilidad de caer en el peor caso. Entre las ventajas de Quicksort también se encuentra un buen aprovechamiento del hardware, dado que durante la partición los accesos al arreglo son secuenciales, lo que favorece la localidad espacial y, probablemente, el uso de la caché (LaMarca & Ladner, 1999).

Conclusiones

En el campo de la computación son innumerables los algoritmos y estructuras que permiten mantener un orden determinado entre los elementos almacenados. Cada uno presenta ventajas y desventajas que dependen de múltiples factores, como la complejidad algorítmica, el uso de memoria, la optimización del hardware y la naturaleza de los datos. Por ejemplo, un árbol balanceado resulta más conveniente en escenarios donde se requiere un flujo constante de información, como en el manejo de una base de datos (Cormen et al., 2022). Por su parte, Heapsort suele usarse por su alta fiabilidad, con un rendimiento que no depende de la naturaleza de los datos, lo que evita ataques que puedan provocar el peor caso $O(n^2)$ de Quicksort (Musser, 1997), o cuando la memoria RAM es limitada, como en sistemas embebidos. Finalmente,

Quicksort suele usarse por su alta velocidad, apoyado en diferentes esquemas de partición para evitar casos degenerados.

Para el problema planteado, que consiste en ordenar una sola vez 100.000 palabras sin repetidos y en orden aleatorio, el enfoque más apropiado es Quicksort: según las mediciones, fue aproximadamente 1,9 veces más rápido que Heapsort y 2,5 veces más rápido que el árbol AVL, con un consumo de memoria menor que el del árbol y una variabilidad menor que la de los otros dos algoritmos. Estos resultados tienen limitaciones: se obtuvieron en una sola máquina, con un solo conjunto de datos, a partir de solo 5 mediciones, sin elementos duplicados y con orden aleatorio, y no se midieron directamente los fallos de caché ni el número de comparaciones, por lo que las explicaciones sobre la caché son hipótesis razonables y no resultados comprobados.

Incluso, existen algoritmos de ordenamiento que combinan varios métodos para tomar lo mejor de cada uno y brindar el máximo rendimiento, como *introsort* (Musser, 1997), que combina Quicksort, Heapsort e *insertion sort* y es el que emplea generalmente la función `std::sort` de la biblioteca estándar de C++. Como programadores, debemos conocer las múltiples alternativas junto con sus características para poder brindar soluciones óptimas.

Referencias

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4.ª ed.). MIT Press.

LaMarca, A., & Ladner, R. E. (1999). The influence of caches on the performance of sorting. *Journal of Algorithms, 31*(1), 66–104. https://doi.org/10.1006/jagm.1998.0985

Musser, D. R. (1997). Introspective sorting and selection algorithms. *Software: Practice and Experience, 27*(8), 983–993. https://doi.org/10.1002/(SICI)1097-024X(199708)27:8<983::AID-SPE117>3.0.CO;2-%23

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4.ª ed.). Addison-Wesley.